

Discover Ansible

✔ Check

Checkpoint: do a commit and call us to check your results (don't stay blocked on a checkpoint if we are busy, we can check ½ checkpoints at the same time)

❓ Question

Point to document/report

💡 Tip

Interesting information

Table of contents

Goals

Introduction

Inventories

Facts

Playbooks

First playbook

Advanced Playbook

Deploy your App

Continuous Deployment

Front

Going Further

Goals

Install and deploy your application automatically with ansible.

Introduction

Inventories

By default, Ansible's inventory is saved in the location `/etc/ansible/hosts` where you already defined your server.

The headings between brackets (eg: [webservers]) are used to group sets of hosts together, they are called, surprisingly, groups. You could regroup them by roles like database servers, front-ends, reverse proxies, build servers...

Let's create a project specific inventory, in your project create an ansible directory, then create a new directory called inventories and in this folder a new file (`my-project/ansible/inventories/setup.yml`):

```
all:
  vars:
    ansible_user: admin
    ansible_ssh_private_key_file: /path/to/private/key
  children:
    prod:
      hosts: hostname or IP
```

Test your inventory with the ping command:

```
ansible all -i inventories/setup.yml -m ping
```

Facts

Let's get information about hosts: these kinds of variables, not set by the user but discovered are called **facts**.

Facts are prefixed by `ansible_` and represent information derived from speaking with your remote systems.

You will request your server to get your OS distribution, thanks to the setup module.

```
ansible all -i inventories/setup.yml -m setup -a "filter=ansible_distribution*" --be
```

Earlier you installed Apache2 server on your machine, let's remove it:

```
ansible all -i inventories/setup.yml -m apt -a "name=apache2 state=absent" --be
```

With Ansible, you just describe the state of your server and let ansible automatically update it for you.

If you run this command another time you won't have the same output as apache2 would have been removed.

❓ Question

3-1 Document your inventory and base commands

Playbooks

First playbook

Let's create a first very simple playbook in `my-project/ansible/playbook.yml`:

```
- hosts: all
  gather_facts: false
  become: true

  tasks:
    - name: Test connection
      ping:
```

Just execute your playbook:

```
ansible-playbook -i inventories/setup.yml playbook.yml
```

You can check your playbooks before playing them using the option: `--syntax-check`

Advanced Playbook

Let's create a playbook to install docker on your server, follow the documentation and create the corresponding tasks: <https://docs.docker.com/engine/install/debian/#install-using-the-repository>.

```
- hosts: all
  gather_facts: true
  become: true

  tasks:
    # Install prerequisites for Docker
    - name: Install required packages
      apt:
        name:
          - apt-transport-https
          - ca-certificates
          - curl
          - gnupg
          - lsb-release
          - python3-venv
        state: latest
        update_cache: yes

    # Add Docker's official GPG key
    - name: Add Docker GPG key
      apt_key:
        url: https://download.docker.com/linux/debian/gpg
        state: present

    # Set up the Docker stable repository
    - name: Add Docker APT repository
      apt_repository:
        repo: "deb [arch=amd64] https://download.docker.com/linux/debian {{ ansible
        state: present
        update_cache: yes

    # Install Docker
    - name: Install Docker
      apt:
        name: docker-ce
        state: present

    # Install Python3 and pip3
    - name: Install Python3 and pip3
      apt:
        name:
          - python3
          - python3-pip
        state: present

    # Create a virtual environment for Python packages
    - name: Create a virtual environment for Docker SDK
      command: python3 -m venv /opt/docker_venv
      args:
        creates: /opt/docker_venv # Only runs if this directory doesn't exist

    # Install Docker SDK for Python in the virtual environment
    - name: Install Docker SDK for Python in virtual environment
      command: /opt/docker_venv/bin/pip install docker

    # Ensure Docker is running
    - name: Make sure Docker is running
      service:
        name: docker
        state: started
        tags: docker
```

Good news, we now have docker installed on our server. One task was created to be sure docker was running, you could check this with an ad-hoc command or by connecting to the server until you really trust ansible.

✔ Check

Docker installed on remote server

Using roles

Our docker install playbook is nice and all but it will be cleaner to have in a specific place, in a role for example. Create a docker role and move the installation task there:

```
ansible-galaxy init roles/docker
```

Call the docker role from your playbook to check your refactor and your installation.

Initialized role has a couple of directories, keep only the one you will need:

- tasks - contains the main list of tasks to be executed by the role.
- handlers - contains handlers, which may be used by this role or outside.

❓ Question

3-2 Document your playbook

Deploy your App

Time has come to deploy your application to your Ansible managed server.

Create specific roles for each part of your application and use the Ansible module: `docker_container` to start your dockerized application. Here is what a `docker_container` task should look like:

```
- name: Run HTTPD
  docker_container:
    name: httpd
    image: your_image_name_from_DockerHub
```

You must have at least this roles :

- install docker
- create network
- launch database
- launch app
- launch proxy

📌 Note

- You will need to add env variables on app and database tasks. Ansible is able to modify the variables either in the `.env` for the db or in the application.yml for the app.
- Don't forget to use existing module for when creating the network
- Don't forget to use the right python interpreter when creating the docker network (refer to `ansible_python_interpreter` variable usage)

📌 Link

- [docker_container module documentation](#)
- [docker_network module documentation](#)

✔ Check

You should be able to access your API on your server.

❓ Question

3-3 Document your docker_container tasks configuration.

Continuous Deployment

Now that you've deployed you database, api and proxy manually on your remote server once, integrate what you've done to you GitHub Actions workflow.

This way your application should be automatically deployed everytime a new image is uploaded to DockerHub.

❓ Question

Is it really safe to deploy automatically every new image on the hub ? explain. What can I do to make it more secure?

✔ Check

Continuous deployment works!

Front

Now that you are able to access your api through your server, it is time to add an actual front-end to display informations coming from your API.

Your database, api and httpd must be up on your server and deployed with your Github Actions.

You can find the code of the **front ready**.

You have to customize your httpd server to make the redirection correct between the API and the front. The httpd server is a proxy within your system.

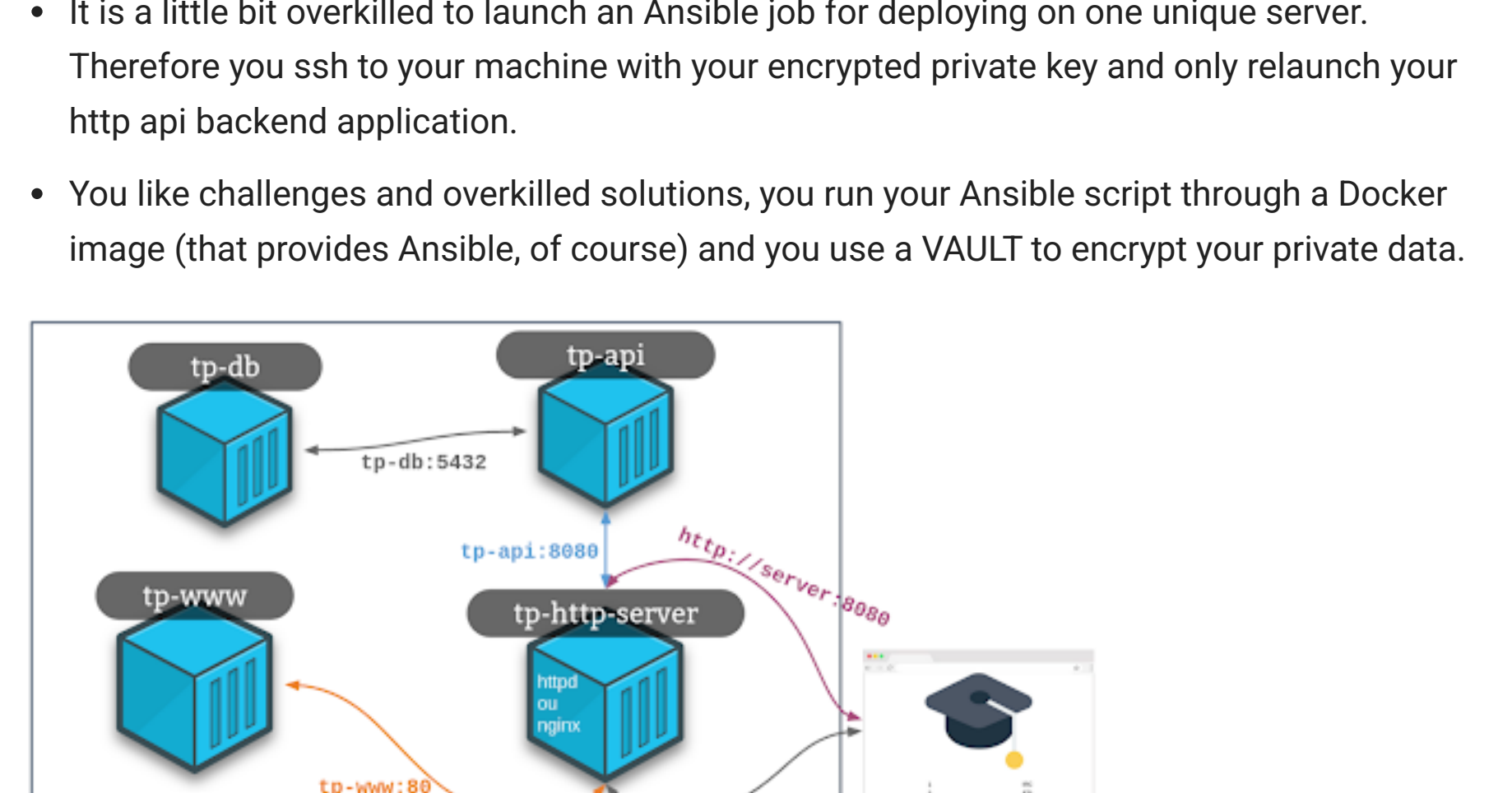
And don't forget to adapt your CI/CD to also handles the front-end.

✔ Check

Front working

Going Further

- It is a little bit overkilled to launch an Ansible job for deploying on one unique server. Therefore you ssh to your machine with your encrypted private key and only relaunch your http api backend application.
- You like challenges and overkilled solutions, you run your Ansible script through a Docker image (that provides Ansible, of course) and you use a VAULT to encrypt your private data.



✔ Check

Continuous deployment with a vault.